

Infinity Arcade Game Submission Requirements

Use this as the hard checklist before submitting a game to Infinity Arcade. If you are using AI to build the game, paste the **AI Builder Prompt** section into your AI tool.

For the full builder eligibility standard, including the universal Arcade shell **Start** overlay requirement, see `docs/runbooks/infinity-arcade-builder-eligibility-standard.md`.

Submission paths

- **The Back:** submit a playable browser game URL. Use `https://` whenever possible.
- **Showroom / on-chain:** submit game details first, then provide a review zip for final on-chain upload.

Hard requirements

- The game must be playable in a browser.
- The game must run inside an iframe.
- The game must work in Chromium browsers and Firefox.
- If marked mobile-compatible, it must work on mobile/tablet too.
- Thumbnail/image uploads must be **2 MB max**.
- Game scores must be positive integers. Higher score must mean better score.
- Do not submit floats, strings, negative scores, or formatted score text.
- Do not make backend/canister/wallet calls inside the game loop.
- Economy actions belong at session boundaries only: start game, game over, submit score, claim/withdraw outside gameplay.
- Ticket games must clearly support score-based ticket payout rules and backed-pool limits.
- Normal games must not pretend to be ticket-backed games.
- No malicious, hidden, phishing, wallet-draining, auto-download, exploit, or obfuscated behavior.

Showroom / on-chain zip requirements

- Package the game as a `.zip`.

- The zip must include an `index.html` entry point.
- Zip size must be **10 MB max**.
- Every file inside the zip must be **2 MB max uncompressed**.
- The game must be self-contained: include required HTML, CSS, JS, images, fonts, sounds, and assets in the zip.
- Do not depend on external runtime asset loading. Avoid external `fetch()`, CDN scripts, remote images, remote audio, external APIs, or server-only logic.

Required submission info

- Game name
- Developer/studio name
- Short description
- Thumbnail image
- Category: `AI Made`, `Independent`, or `Studio`
- Platform: `Desktop`, `Desktop + Mobile`, or equivalent
- Game type: ticket game or regular game
- Token cost, if applicable: **1–20 Tokens**
- Scoring mode: `best-of` or `cumulative`
- Purchase price, if applicable: **0–1000 Tokens**
- For The Back: playable game URL
- For Showroom/on-chain: review/demo URL if available, plus the zip after review request

Optional/admin-assisted:

- Screenshots: up to **5**, recommended 16:9 and at least 800px wide
- Hosted thumbnail URL, if already available

Input requirements

The game should support both:

- Native keyboard input inside the iframe.
- Infinity Arcade parent input messages:

```
js
{
```

```

type: 'arcade-key',
player: 0,
action: 'left' | 'right' | 'up' | 'down' | 'a' | 'b' | 'c' | 'd' | 'start',
state: 'down' | 'up'
}

```

Rules:

- Map movement to `left/right/up/down` .
- Map `a` and/or `start` to primary action/start/confirm.
- Use `b/c/d` only for secondary actions.
- Track held keys correctly; only treat an action as “just pressed” on an up-to-down transition.
- Ignore unknown actions safely.
- Keep native keyboard support even if `arcade-key` support is added.

Universal Start overlay compatibility

Infinity Arcade Showroom should provide a platform-level **Start** button overlay for every launched game, including regular games and ticket-payout games.

This overlay is owned by the Arcade shell. Builders should not rely on custom focus hacks or browser-specific click behavior. To benefit from the universal Start overlay, the game must treat Arcade `arcade-key` messages with `action: 'a'` and/or `action: 'start'` as Start/Confirm when the game is waiting to begin.

The shell overlay can focus an iframe and send the standard input message, but it cannot reliably start a game that ignores the Arcade input contract.

Packaging rules for Showroom zip

Recommended zip layout:

```

text
index.html
assets/
  sprites.png
  music.ogg
  sfx.ogg
  font.woff2

```

Avoid:

```
text
__MACOSX/
.DS_Store
.git/
node_modules/
src-only files with no built output
files over 2 MB
zip over 10 MB
```

Audio tips:

- Prefer OGG/MP3 loops and compressed SFX.
- Avoid large WAV files.
- Procedural audio is strongly preferred when possible.

Minimum pre-submit test

Before submission, verify:

- Runs standalone from the submitted URL or local `index.html`.
- Runs embedded in an iframe.
- Works in Chrome/Brave.
- Works in Firefox.
- Keyboard controls work.
- `arcade-key` `postMessage` controls work.
- No external assets are required for Showroom/on-chain mode.
- No canister/backend calls happen every frame.
- Score is submitted only after game over or explicit completion.
- Score is a positive integer.
- Mobile works if you claim mobile support.

AI Builder Prompt

Build a browser game for Infinity Arcade with these constraints:

- Output a complete static web game with `index.html` as the entry point for Showroom/on-chain zip submission.

- The final Showroom zip must be under 10 MB.
- No individual file may exceed 2 MB uncompressed.
- Bundle all assets locally; do not rely on CDNs, remote images, remote audio, external APIs, servers, Node runtime, or runtime package installs.
- The game must run in an iframe in Chrome/Brave and Firefox.
- Implement native keyboard controls.
- Also implement `window.addEventListener('message', ...)` support for Infinity Arcade `arcade-key` messages with actions `left`, `right`, `up`, `down`, `a`, `b`, `c`, `d`, and `start`, with states `down` and `up`.
- Treat `a` and/or `start` as Start/Confirm when the game is waiting to begin, so the Arcade shell's universal Start overlay can activate the game.
- Track input state so held keys do not repeatedly trigger one-shot actions.
- Use `requestAnimationFrame` for the game loop.
- Do not call backend, wallet, ICP, canister, or network APIs inside the frame loop.
- If the game has scoring, score must be a positive integer where higher is better.
- Include a clear game-over state where the final integer score can be submitted by the parent arcade.
- Keep assets small; prefer compressed images, OGG/MP3 audio, or procedural sound.
- Provide final files in a zip-ready structure with no `node_modules`, `.git`, `__MACOSX`, `.DS_Store`, or oversized files.